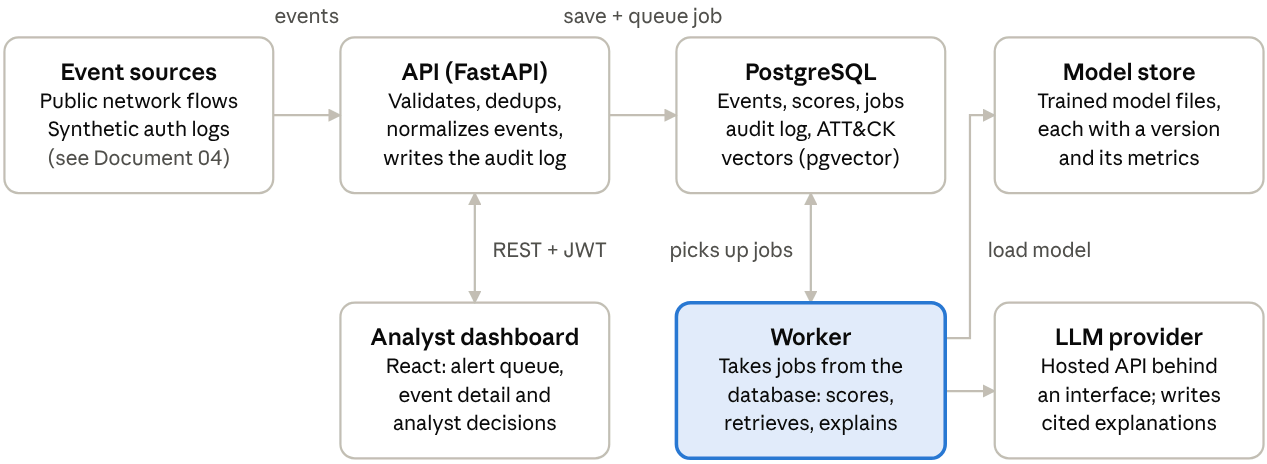
SentinelIQ 07 — Architecture (HLD) & ADRs

Oct 13, 2026 · @maxi

SentinelIQ is a small set of containers around one PostgreSQL database: an API, a background worker, a React frontend, and a hosted LLM behind an interface.

Architecture at a glance

The API stays fast because the worker handles scoring and the LLM



Analysts alone record decisions; the worker never acts on an event.

architecture · 4 containers, 2 external services

The API validates and stores events and queues a job; the worker, in a separate container, does the scoring, retrieval and LLM call, so ingestion stays fast.

Components

Component	Responsibility	Technology
Frontend	Analyst dashboard: queue, event detail, actions, metrics	React
API	Authentication, ingestion, validation, normalization, queries, decisions, audit writes	FastAPI (Python)

Component	Responsibility	Technology
Worker	Takes jobs from the database: scoring, retrieval and explanation generation	Same Python codebase, separate container
Database	Events, scores, explanations, jobs, audit log and ATT&CK knowledge vectors	PostgreSQL with the pgvector extension
Model store	Versioned model files, indexed by the `model_versions` table	A mounted volume locally; cloud storage in production
LLM provider	Writes explanations from retrieved context only	Hosted API behind a `LlmClient` interface

Request and data flow

The path of one event, from a source to an analyst's decision:

1. A source sends the event to `POST /api/v1/events` with a JWT.
2. The API validates the payload, derives the deduplication key, normalizes the event and writes it to `events`; a duplicate is recognized and not stored twice.
3. In the same database transaction the API inserts a `jobs` row and returns immediately, which keeps ingestion fast (NFR-P1).
4. The worker claims the job using row locking, so two workers never take the same job, scores the event, and stores the score with its model version.
5. If the score is at or above the threshold, the worker retrieves ATT&CK chunks, calls the LLM through `LlmClient`, validates the answer and stores the explanation.
6. The dashboard reads the queue from the API; the analyst's decision is written to `audit_log` with a required note.

Environments

Environment	How it runs	Data and secrets
Local	`docker-compose` starts the API, worker, database and frontend with one command	Seed data from Document 04; secrets in a local environment file that is never committed
CI	GitHub Actions with a PostgreSQL service container	Test fixtures and recorded LLM responses; no real secrets
Production (Phase 7)	The same images on Google Cloud Run, with a managed PostgreSQL that supports pgvector (to be confirmed in Phase 7)	Same public and synthetic data; secrets in a secret manager

The worker polls for jobs, so in production it runs as its own always-on service. The exact Cloud Run settings are decided in Phase 7.

Architecture decision records

Each row records one decision, what else was considered, and why this option won. A changed decision gets a new row that supersedes the old one.

ID	Decision	Alternatives considered	Reason
ADR-001	FastAPI for the backend	Django REST Framework, Flask	Built-in request validation, generated OpenAPI document, and async support for I/O-heavy ingestion
ADR-002	PostgreSQL as the only datastore	A separate document store or search engine	Relational integrity for the audit trail, JSON columns for raw events, and one system to run

ID	Decision	Alternatives considered	Reason
ADR-003	A job table in PostgreSQL with a separate worker	Redis with Celery, a cloud message queue	Fewer moving parts, and the job is created in the same transaction as the event; revisit with a broker (SH1) only if load tests show the table is the bottleneck
ADR-004	pgvector for retrieval	A dedicated vector database	The ATT&CK corpus is small, and keeping vectors beside the data avoids another service
ADR-005	Retrieval-augmented generation	Fine-tuning an LLM	Cheaper, every claim can cite its source, and updating means re-indexing a new ATT&CK release
ADR-006	Hosted LLM behind a `LlmClient` interface	A locally run model only	Fastest route for v1, and the interface keeps the vendor out of the business logic so a local model can replace it
ADR-007	Gradient-boosted classifier as the primary model	Deep learning, unsupervised detection only	Labeled data exists, scoring is fast, and feature importances aid explanation (details in Document 09)
ADR-008	React single-page frontend	Server-rendered templates	An interactive queue with filters and detail panels
ADR-009	Short-lived JWT access tokens	Server-side sessions	Stateless API containers, and short expiry limits the damage of a stolen token

Scaling path

These are design intentions, not measured results. The Phase 7 load test shows which limit appears first.

Pressure at 100 times the load	First response	Later response
Ingestion rate	Run more stateless API instances; encourage the batch endpoint	Partition the events table by time
Scoring backlog	Add worker instances; row locking lets several run together	Move to a message broker (SH1) and a separate model-serving service
LLM cost and delay	Explain only events above the threshold; cache results; cap daily calls	A smaller model for a first pass; batched calls
Queue queries on a large table	Indexes on risk and status; archive old events	Time partitions and a read replica for the dashboard
Metrics page	Cursor pagination	Cache the summary figures